جامعة الملك سعود
King Saud University

# CSC311 Project

*Shahad Almutairi 442201347*
*Lujain ALSuleiman 443200947*
*Bushra Alkhudairi 443200979*

*2024/24/02*
*INSTRUCTORS:*
*Dr. Sarab Almuhaideb*
*Dr. Malak Almojaly*

# Contents

# Pseudocode of bruteForce Algorithm

```
Algorithm bruteForce():
   stack = new Stack()
   stack.push(new State(0, 0, []))

   while stack is not empty:
      currentState = stack.pop()
      currentIndex = currentState.currentIndex
      currentRisk = currentState.currentRisk
      currentAllocation = currentState.currentAllocation

      if currentIndex is equal to the number of assets:
         totalQuantity = sum of quantities in currentAllocation
         if currentRisk is less than or equal to maxRisk and totalQuantity is less
than or equal to maxInvestment:
            currentReturn = sum of (expectedReturn * quantity) in
currentAllocation
            if currentReturn is greater than maxReturn:
               maxReturn = currentReturn
               optimalAllocation = copy of currentAllocation

      else:
         currentAsset = assets[currentIndex]
         currentQuantity = currentAsset.quantity

         for i from 0 to currentQuantity:
            clonedAsset = new Asset(currentAsset.id, currentAsset.expectedReturn,
currentAsset.riskLevel, i)
            clonedAllocation = copy of currentAllocation
            clonedAllocation.add(clonedAsset)

            weight = i / maxInvestment
            stack.push(new State(currentIndex + 1, currentRisk +
currentAsset.riskLevel * weight, clonedAllocation))
```

# The time complexity of bruteForce Algorithm

*For each portfolio the algorithm considers m^n allocations, where m is the number of possible quantities for one asset and n is the number of asset, thus the number of allocations the algorithm explores grows exponentially with the number of assets, resulting in Θ(m^n)*

| *Best case* | *Worst case* |
|---|---|
| *The best-case time complexity of this algorithm is when the portfolio consist of one asset with 0 quantity:*<br>*A : 0.02 : 0.03 : 0*<br>*In this case there is 1 asset with 1 possible quantity leading to time complexity of Ω(1)* | *The worst-case time complexity of this algorithm is exponential O(m^n), where (n) is the number of assets and (m) is the number of possible quantities for the asset.*<br>*This can be demonstrated with an example where the algorithm explores all possible combinations of assets. Consider a scenario*<br>*A : 0.02 : 0.03 : 1*<br>*B : 0.07 : 0.04 : 1*<br>*C : 0.5 : 0.012 : 1*<br>*with three assets, each with two possible quantities (0 , 1). This results in 2^3 = 8 possible combinations (A,B,C):*<br>*1. (1,1,0)*<br>*2. (1,1,1)*<br>*3. (1,0,0)*<br>*4. (1,0,1)*<br>*5. (0,1,0)*<br>*6. (0,1,1)*<br>*7. (0,0,0)*<br>*8. (0,0,1)*<br>*The algorithm would explore all 8 combinations, the algorithm performs operations such as cloning assets, calculating risk, and checking constraints, leading to an exponential number of operations.* |

## Experimental results demonstrating the performance of the algorithm:

*Example 1:*
*AAPL : 0.05 : 0.02 : 300*
*GOOGL : 0.08 : 0.03 : 500*
*Total investment is 800 units*
*Risk tolerance level is 0.024*

*Result:*
*Optimal Allocation:*
*AAPL: 210 units*
*GOOGL: 500 units*
*Expected Portfolio Return: 0.0505*
*Portfolio Risk Level: 0.024*

*Example 2:*
*GOOGL : 0.07 : 0.04 : 600*
*TSLA : 0.1 : 0.05 : 600*
*Total investment is 1200 units*
*Risk tolerance level is 0.02*

*Result:*
*Optimal Allocation:*
*GOOGL: 1 units*
*TSLA: 479 units*
*Expected Portfolio Return: 0.04797*
*Portfolio Risk Level: 0.02*

*Example 3:*
*APPL : 0.07 : 0.04 : 200*
*FB : 0.1 : 0.05 : 150*
*Total investment is 300 units*
*Risk tolerance level is 0.032*

*Result:*
*Optimal Allocation:*
*APPL: 55 units*
*FB: 148 units*
*Expected Portfolio Return: 0.01865*
*Portfolio Risk Level: 0.032*

# Screenshots of complete code and sample run

```java
import java.io.BufferedReader;
import java.io.LineNumberReader;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Stack;
import java.util.stream.Stream;
public class BruteForcePortfolioAllocation {
    static List<Asset> assets;
    static List<Asset> optimalAllocation;
    static double maxRisk;
    static double maxInvestment;
    static double maxReturn;

    Run | Debug
    public static void main(String[] args) {
        readInput(fileName:"Example3.txt");
        bruteForce();
        writeOutput(fileName:"Output_BruteForce.txt");
    }
```

```java
static void bruteForce() {
    Stack<State> stack = new Stack<>();
    stack.push(new State(currentIndex:0, currentRisk:0, new ArrayList<>()));
    while (!stack.isEmpty()) {
        State currentState = stack.pop();
        int currentIndex = currentState.currentIndex;
        double currentRisk = currentState.currentRisk;
        List<Asset> currentAllocation = currentState.currentAllocation;

        if (currentIndex == assets.size()) {
            double totalQuantity = currentAllocation.stream().mapToDouble(a -> a.quantity).sum();
            if (currentRisk <= maxRisk && totalQuantity <= maxInvestment) {
                double currentReturn = currentAllocation.stream().mapToDouble(a -> (a.expectedReturn * a.quantity)/1000).sum();
                if (currentReturn > maxReturn) {
                    maxReturn = currentReturn;
                    optimalAllocation = new ArrayList<>(currentAllocation);
                }
            }
        } else {
            Asset currentAsset = assets.get(currentIndex);
            int currentQuantity = currentAsset.quantity;
            for (int i = 0; i <= currentQuantity; i++) {
                Asset clonedAsset = new Asset(currentAsset.id, currentAsset.expectedReturn, currentAsset.riskLevel, i);
                List<Asset> clonedAllocation = new ArrayList<>(currentAllocation);
                clonedAllocation.add(clonedAsset);
                double weight = i/maxInvestment;
                stack.push(new State(currentIndex + 1, currentRisk + currentAsset.riskLevel * weight, clonedAllocation));
            }
        }
    }
}

static void readInput(String fileName) {
    long noOfLines = -1;
    try (Stream<String> fileStream = Files.lines(Paths.get(fileName))) {
        noOfLines = (int) fileStream.count();
    }
    catch (IOException e) {
        e.printStackTrace();
    }
    try (BufferedReader br = new BufferedReader(new FileReader(fileName))) {
        assets = new ArrayList<>();
        String line;
        long counter = 0;
        while ((line = br.readLine()) != null) {
            if (counter<noOfLines-2) {
                String[] parts = line.split(regex:":");
                String id = parts[0].trim();
                double expectedReturn = Double.parseDouble(parts[1].trim());
                double riskLevel = Double.parseDouble(parts[2].trim());
                int quantity = Integer.parseInt(parts[3].trim());
                assets.add(new Asset(id, expectedReturn, riskLevel, quantity));
            }
            else {
                String[] parts2 = line.split(regex:"\\s+");
                if((parts2[0]).equalsIgnoreCase(anotherString:"Total")) {
                    maxInvestment = Double.parseDouble(parts2[3].trim());
                }
                if((parts2[0]).equalsIgnoreCase(anotherString:"Risk")) {
                    maxRisk = Double.parseDouble(parts2[4].trim());
                }
            }
            counter++;
        }
    }
    catch (IOException e) {
        e.printStackTrace();
    }
}
```

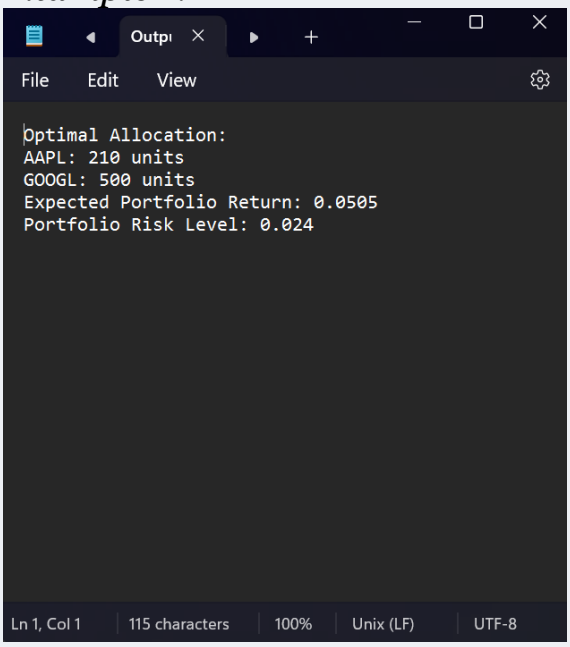
```
    static void writeOutput(String fileName) {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(fileName))) {
            bw.write(str:"Optimal Allocation:\n");
            for (Asset asset : optimalAllocation) {
                bw.write(asset.id + ": " + asset.quantity + " units\n");
            }
            bw.write("Expected Portfolio Return: " + maxReturn + "\n");
            bw.write("Portfolio Risk Level: " + maxRisk + "\n");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    static class State {
        int currentIndex;
        double currentRisk;
        List<Asset> currentAllocation;

        public State(int currentIndex, double currentRisk, List<Asset> currentAllocation) {
            this.currentIndex = currentIndex;
            this.currentRisk = currentRisk;
            this.currentAllocation = currentAllocation;
        }
    }
}
```

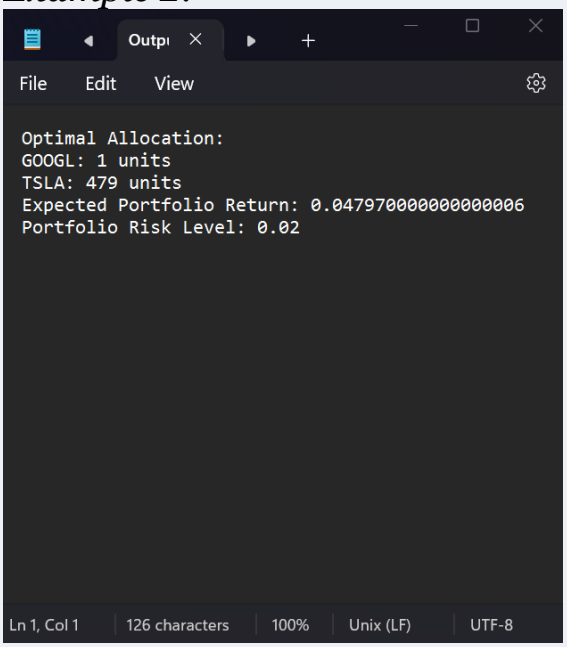
```
public class Asset {
    String id;
    double expectedReturn;
    double riskLevel;
    int quantity;

    public Asset(String id, double expectedReturn, double riskLevel, int quantity) {
        this.id = id;
        this.expectedReturn = expectedReturn;
        this.riskLevel = riskLevel;
        this.quantity = quantity;
    }
}
```

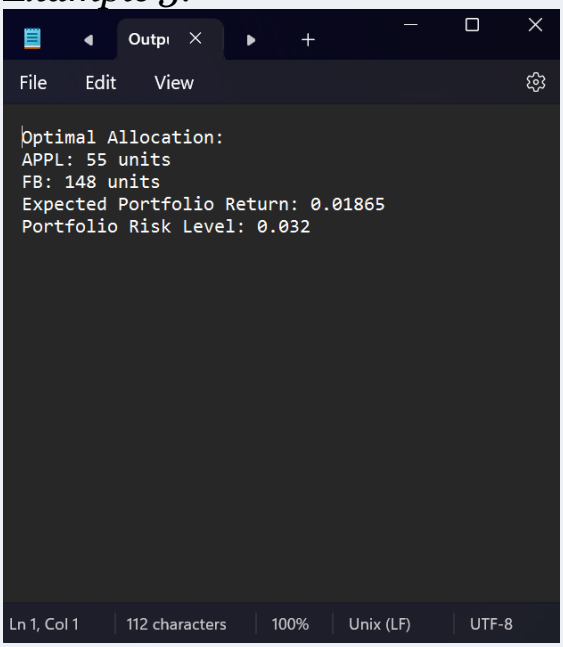
*Example 1:*



*Example 2:*




*Example 3:*



# Source code

CSC311ProjectPhase1SourceCode.zip

# Challenges

*Let's discuss the challenges we encountered while developing this portfolio optimization code and how we addressed them collaboratively:*

*One significant challenge we grappled with was the computational complexity of the brute-force algorithm. As the number of assets in the portfolio increases, the algorithm's runtime could become impractical. To mitigate this, we employed a stack-based approach for efficient state space exploration. However, we're aware that further optimization may be necessary, and we should explore advanced techniques, such as dynamic programming, to enhance efficiency, especially for larger portfolios.*

*Precision in floating-point arithmetic was another hurdle. Given the nature of risk and return calculations, rounding errors could potentially affect the accuracy of results. We need to be diligent in handling numerical precision issues to ensure that the computed values align with our expectations. This might involve considering alternative data types or approaches that mitigate precision concerns.*

*Input validation emerged as a crucial aspect during development. The code reads input from a text file, and any inconsistencies or format discrepancies could lead to unexpected behavior. Implementing robust error handling and validation mechanisms for input data is essential to enhance the reliability of the code, making it more resilient to variations in input formats.*

*Lastly, while the code structure is clear, we identified the need for more comprehensive comments and documentation. Providing explicit explanations for algorithmic steps, variable meanings, and overall code structure will contribute to improved readability and facilitate easier collaboration among team members.*

*Moving forward, let's continue to iterate on these aspects collaboratively to ensure our portfolio optimization code not only meets our current requirements but also remains flexible and scalable for future enhancements.*

# Evaluation Rubric

| Part 1: Team Work | | | |
|---|---|---|---|
| Criteria | Student1:Lujain | Student2:Shahad | Student3:Bushra |
| Work division: Contributed equally to the work | | | |
| Student succeeds in smoothly | | | |
| Peer evaluation: Level of commitments (Interactivity with other team members), and professional behavior towards team & TA | | | |
| Project Discussion: Accurate answers, understanding of the presented work, good listeners to questions | | | |
| Time management: Attending on time, being ready to start the demo, good time management in discussion and demo. | | | |
| Total/3 | | | |

# THANK YOU